StockPay API Reference

Accept UPI payments in INR through a hosted payment page. REST + JSON, server-to-server.

Base URL Authentication Create payment Get payment Statuses Test mode Currencies Webhooks Errors Node.js example

Base URL

```
https://stockpay-in.vercel.app/api/v1
```

The **same base URL** is used for test (sandbox) and live. The API key decides the mode: `sk_test_…` creates test payments, `sk_live_…` creates real ones. Test and live payments are kept apart — a key can only see payments made in its own mode.

Authentication

Send your secret key as a Bearer token on every request. There is no separate merchant ID or client ID — the key identifies your account.

```
Authorization: Bearer sk_test_xxxxxxxxxxxxxxx
Content-Type: application/json
```

Keys are in the StockPay admin panel → **API Keys**. Keep them on your server only (never in a mobile app or browser code). Missing/invalid key → `401 unauthorized`.

Create a payment (hosted payment page)

POST `/orders`

FIELD	TYPE	DESCRIPTION
`amount` **required**	number | string	Amount in rupees, e.g. `499` or `"499.00"`. Minimum is set by the merchant (currently ₹100). Max ₹99,999.
`currency`	string	Optional. Only `"INR"` is accepted; anything else → `400 unsupported_currency`.
`reference`	string ≤ 80	Your order ID. Idempotent: sending the same reference again returns the existing pending/paid payment instead of a new one.

`customer`	object	Optional `{ "name", "phone", "email" }`.
`note`	string ≤ 200	Optional, shown on the payment page.
`return_url`	https URL	Where the customer is sent after paying. We append `?order_id=…&reference=…&status=…` — do not trust these values, confirm with Get payment or the webhook.

Request

```
curl -X POST https://stockpay-in.vercel.app/api/v1/orders \
  -H "Authorization: Bearer sk_test_xxx" \
  -H "Content-Type: application/json" \
  -d '{ "amount": 499, "currency": "INR", "reference": "ORD-1001",
        "customer": { "name": "Rahul", "email": "rahul@example.com" },
        "return_url": "https://yourapp.com/payment/return" }'
```

Response `201 Created`

```
{
  "id": "ord_Ab12Cd34Ef56",
  "reference": "ORD-1001",
  "status": "pending",
  "amount": "499.00",
  "amount_payable": "499.37",
  "amount_paid": null,
  "currency": "INR",
  "utr": null,
  "matched_by": null,
  "customer": { "name": "Rahul", "phone": "", "email": "rahul@example.com" },
  "note": "",
  "payment_url": "https://stockpay-in.vercel.app/pay/ord_Ab12Cd34Ef56",
  "created_at": "2026-10-13T10:00:00.000Z",
  "expires_at": "2026-10-13T10:15:00.000Z",
  "paid_at": null,
  "test": true
}
```

Redirect the customer to `payment_url`. The page expires at `expires_at` (15 minutes by default). In live mode `amount_payable` includes up to ₹0.99 extra so each payment can be identified; the customer pays that exact amount.

Get a payment's status

GET `/orders/{id}` — returns the same object as above.

`GET` `/orders?reference=ORD-1001` — `{ "data": [ … ] }`, newest first.
`POST` `/orders/{id}/cancel` — cancels a pending or expired payment.
Fields to read: `status`, `amount_paid`, `currency`, `paid_at`, `utr` (12-digit UPI reference from the bank).

Statuses

STATUS	MEANING
`pending`	Waiting for the customer to pay.
`paid`	Money received and confirmed. Final.
`expired`	Not paid in time. A late payment may still turn it `paid` (within 24 h).
`cancelled`	Cancelled by you or the merchant. Final.

There is no `failed` status: a UPI attempt that fails never reaches the merchant's bank, so the payment simply stays `pending` until it is paid or expires. Treat `expired` and `cancelled` as "not paid".

Test mode (sandbox)

- Use your `sk_test_…` key with the same base URL.
- Payments are created with `"test": true` and never touch a bank account.
- Open the `payment_url` — instead of a UPI QR it shows a **Simulate successful payment** button. Pressing it marks the payment `paid` and sends the `order.paid` webhook with `"test": true`.
- There are **no test card numbers**: StockPay does not process cards. Payments are UPI only.

Supported currencies and methods

INR only, paid by **UPI** (any UPI app or QR scan). Other currencies, cards, net banking and wallets are **not supported**.

Webhooks

Set your webhook URL in the admin panel → **API Keys**. When a payment is confirmed we send:

```
POST https://yourapp.com/webhooks/stockpay
Content-Type: application/json
X-StockPay-Event: order.paid
X-StockPay-Delivery: 42
X-StockPay-Timestamp: 1791890000
X-StockPay-Signature: sha256=5d1c…

{ "event": "order.paid", "created_at": "…", "order": { …same object as Get payment… }
```

Verify the signature

`signature = "sha256=" + HMAC_SHA256(webhook_secret, timestamp + "." + raw_body)`, hex. Use the **raw** request body, compare in constant time, and reject timestamps older than 5 minutes.

```
const crypto = require('crypto');
function verifyStockPay(rawBody, headers, secret) {
  const ts = headers['x-stockpay-timestamp'];
  const sig = headers['x-stockpay-signature'] || '';
  if (!ts || Math.abs(Date.now() / 1000 - Number(ts)) > 300) return false;
  const expected = 'sha256=' + crypto.createHmac('sha256', secret).update(ts + '.' + r
  return sig.length === expected.length && crypto.timingSafeEqual(Buffer.from(sig), Bu
}
```

Reply with any **2xx within 10 seconds**. Otherwise we retry after 1 min, 5 min, 30 min, 2 h, 6 h, 12 h and 24 h. The same event can arrive more than once — make your handler idempotent (e.g. by `order.id`).

Errors

```
{ "error": { "code": "amount_too_low", "message": "minimum amount is ₹100" } }
```

HTTP	CODE	MEANING
400	`bad_request`	Missing/invalid field.
400	`amount_too_low`	Below the merchant's minimum.
400	`unsupported_currency`	Currency other than INR.
401	`unauthorized`	Missing or invalid API key.
404	`not_found`	Unknown payment ID (or other mode's payment).

409	`reference_conflict`	An open payment with this reference has a different amount.
409	`conflict`	Payment can't be cancelled in its current status.
503	`no_account` / `busy`	Merchant can't accept payments right now — retry later.

Node.js example

```
const BASE = 'https://stockpay-in.vercel.app/api/v1';
const KEY = process.env.STOCKPAY_KEY; // sk_test_… or sk_live_…

async function createPayment(orderId, amountInr) {
  const r = await fetch(BASE + '/orders', {
    method: 'POST',
    headers: { Authorization: 'Bearer ' + KEY, 'Content-Type': 'application/json' },
    body: JSON.stringify({ amount: amountInr, currency: 'INR', reference: orderId,
                           return_url: 'https://yourapp.com/payment/return' }),
  });
  const data = await r.json();
  if (!r.ok) throw new Error(data.error.message);
  return data; // redirect the customer to data.payment_url
}

async function getPayment(id) {
  const r = await fetch(BASE + '/orders/' + id, { headers: { Authorization: 'Bearer '
  return r.json(); // { status: 'paid' | 'pending' | 'expired' | 'cancelled', amount_p
}
```